Chapter 8 — Window Functions: Rankings, Running Totals and Row-to-Row Comparisons

`GROUP BY` answers "what is the total per department?" but throws away the individual rows to do it. Very often you need both at once: *each* employee **and** their department's average; *each* day's sales **and** the running total so far; *each* order **and** the previous order by the same customer. **Window functions** do exactly that. They compute a value across a set of related rows — the *window* — while keeping every row in the result.

They replace awkward self-joins (a table joined to itself) and correlated subqueries (subqueries re-run for every outer row), and they sit behind many SQL interview questions (top-N per group, running totals, second-highest salary, streaks, month-over-month growth).

What you'll learn

#	Topic	You will be able to…
8.1	`OVER()`	explain how a window function differs from `GROUP BY`
8.2	`PARTITION BY`	compute per-group values next to each row (share of total, diff from average)
8.3	`ORDER BY` in a window	build running totals and understand the default frame and its tie trap
8.4	Frames: `ROWS` / `RANGE` / `GROUPS`, `EXCLUDE`	choose exactly which rows each calculation sees
8.5	Ranking	use `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE`, `PERCENT_RANK`, `CUME_DIST` and handle ties
8.6	Value functions	compare rows with `LAG` / `LEAD`, `FIRST_VALUE` / `LAST_VALUE` / `NTH_VALUE`
8.7	Running & moving calculations	running totals, moving averages, cumulative share, growth %
8.8	Top-N per group & dedup (de-duplication: removing extra copies)	the most-asked window pattern, plus "keep the latest row"
8.9	Gaps and islands	find streaks, missing ranges and sessions (bursts of activity separated by idle time)
8.10	Named windows, `FILTER`, evaluation order, performance	write tidy window queries and know how they execute

Setup. The `company` sample database from chapter 1 plus two small tables added in `code/00_schema.sql`:

Table	Rows	Why
`daily_sales(store, day, amount)`	Mumbai 1–7 March 2025; Pune the same week **except 4 March** (closed)	running totals, moving averages, frames, a gap
`logins(user_id, login_date)`	two users' login days in May 2025 with breaks	streaks (gaps and islands)

Mumbai's sales for 3 and 4 March are both 1500, and two Pune days are both 650 — deliberate ties. In `employees`, Rahul and Meera (Engineering) both earn 98 000 and Arjun and Sara (Sales) both earn 68 000.

8.1 What a window function is

What it is

A **window function** is a function that, for each row, looks at a set of rows related to it (its **window**) and returns one value — **without merging the rows**. You recognise it by the `OVER (...)` clause after the function call:

```
function_name(args) OVER (
    [PARTITION BY ...]      -- split rows into independent groups
    [ORDER BY ...]          -- order rows inside each group
    [frame clause]          -- which rows around the current one to use
)
```

Analogy: in a class photo, `GROUP BY` replaces each row of students with one sign ("Row 2: average height 150 cm"). A window function keeps every student and pins a badge on each: "your row's average is 150 cm", "you are 3rd tallest in your row", "the student to your left is 2 cm shorter".

The problem it solves

Compare `GROUP BY` with a window:

```
-- code/01_window_basics.sql
SELECT dept_id, round(avg(salary)) AS dept_avg
FROM employees WHERE dept_id IN (1, 2)
GROUP BY dept_id ORDER BY dept_id;
```

Output:

```
 dept_id | dept_avg
---------+----------
       1 |   130750
       2 |    89500
(2 rows)
```

Two rows: the individual employees are gone. With windows, every employee stays and gets the aggregates beside them:

```
SELECT first_name, dept_id, salary,
       round(avg(salary) OVER ())                    AS company_avg,
       round(avg(salary) OVER (PARTITION BY dept_id)) AS dept_avg,
       count(*)          OVER (PARTITION BY dept_id)  AS dept_size
FROM employees
WHERE dept_id IN (1, 2)
ORDER BY dept_id, salary DESC;
```

Output:

```
 first_name | dept_id |  salary   | company_avg | dept_avg | dept_size
------------+---------+-----------+-------------+----------+-----------
 Vikram     |       1 | 185000.00 |      110125 |   130750 |         4
 Neha       |       1 | 142000.00 |      110125 |   130750 |         4
 ...
 Daniel     |       2 | 150000.00 |      110125 |    89500 |         4
 Priya      |       2 |  72000.00 |      110125 |    89500 |         4
 ...
(8 rows)
```

- `OVER ()` — an empty window means "all rows of the result" (here the 8 rows left after `WHERE`), so `company_avg` is the same on every row.
- `OVER (PARTITION BY dept_id)` — the window is "all rows with my `dept_id`".

Without window functions you'd need a join to a grouped subquery (or a correlated subquery per row) for each extra column.

How it works

1. The query's `FROM`, `WHERE`, `GROUP BY` and `HAVING` run first and produce a set of rows.
2. For each window function, rows are split into **partitions** (`PARTITION BY`) and sorted inside each partition (`ORDER BY`).
3. For each row, the function looks at the rows of its window (the whole partition, or a **frame** within it — 8.3/8.4) and computes one value.
4. That value becomes a new column. Row count is unchanged.
5. Then `DISTINCT`, the final `ORDER BY` and `LIMIT` run.

Two families of functions can be used with `OVER`:

Family	Functions	Notes
Aggregate functions used as windows	`sum`, `avg`, `count`, `min`, `max`, `string_agg`, `array_agg`, `bool_and`, …	any normal aggregate becomes a window function when you add `OVER`
Dedicated window functions	`row_number`, `rank`, `dense_rank`, `percent_rank`, `cume_dist`, `ntile`, `lag`, `lead`, `first_value`, `last_value`, `nth_value`	only valid with `OVER`; the ranking ones take no arguments

Dialect notes

	Window functions available since	Frame support
PostgreSQL	8.4 (2009)	`ROWS`, `RANGE` with offsets, `GROUPS`, `EXCLUDE` (offsets/`GROUPS`/`EXCLUDE` since 11)
MySQL	8.0 (2018)	`ROWS`, `RANGE`; no `GROUPS`, no `EXCLUDE`
SQLite	3.25 (2018)	`ROWS`, `RANGE`; `GROUPS` and `EXCLUDE` since 3.28
SQL Server	ranking 2005; aggregate `ORDER BY` + frames 2012	`ROWS`; `RANGE` only with `UNBOUNDED`/`CURRENT ROW`; no `GROUPS`/`EXCLUDE`
Oracle	8i (1999)	full, plus extensions

TL;DR

`f(...) OVER (...)` computes across related rows but keeps every row. `OVER ()` = all rows; add `PARTITION BY`, `ORDER BY` and a frame to narrow it.

8.2 PARTITION BY: per-group values next to each row

What it is

`PARTITION BY expr` splits the rows into independent groups, like `GROUP BY` does, but without collapsing them. Each row's window is its own partition; the function "restarts" in every partition.

Typical uses

```sql
SELECT first_name, dept_id, salary,
       round(100.0 * salary / sum(salary) OVER (PARTITION BY dept_id), 1) AS pct_of_dept_payroll,
       salary - round(avg(salary) OVER (PARTITION BY dept_id))            AS vs_dept_avg
FROM employees
WHERE dept_id IN (1, 2)
ORDER BY dept_id, salary DESC;
```

Output:

```
 first_name | dept_id |  salary   | pct_of_dept_payroll | vs_dept_avg
------------+---------+-----------+---------------------+-------------
 Vikram     |       1 | 185000.00 |                35.4 |    54250.00
 Neha       |       1 | 142000.00 |                27.2 |    11250.00
 Rahul      |       1 |  98000.00 |                18.7 |   -32750.00
 Meera      |       1 |  98000.00 |                18.7 |   -32750.00
 ...
(8 rows)
```

"Percent of group total" and "difference from group average" are the two most common `PARTITION BY` calculations. `100.0 *` forces decimal division (chapter 3).

Partitioning rules

- You can partition by several expressions: `PARTITION BY store, date_trunc('month', day)`.
- `NULL` values form **one** partition together (just like `GROUP BY`):

```sql
SELECT first_name, dept_id, count(*) OVER (PARTITION BY dept_id) AS same_dept
FROM employees
WHERE dept_id IS NULL OR dept_id = 4
ORDER BY dept_id NULLS LAST;
```

Output:

```
 first_name | dept_id | same_dept
------------+---------+-----------
 Kiran      |       4 |         2
 Omar       |       4 |         2
 Asha       |  (null) |         2
 Rohan      |  (null) |         2
(4 rows)
```

- Different window functions in one `SELECT` can use different partitions (the first example mixed `OVER ()` with `OVER (PARTITION BY dept_id)`).

Common mistakes

- **Expecting fewer rows.** A window never removes rows; if you want one row per group, you want `GROUP BY` (or filter afterwards, 8.8).
- **Mixing a window and a non-grouped column in a `GROUP BY` query**: in a grouped query, window functions see the *grouped* rows, so their arguments must be grouped columns or aggregates (8.7 shows `sum(sum(x)) OVER ()`).

Self-check

1. *How many rows does `SELECT name, avg(salary) OVER (PARTITION BY dept_id) FROM employees` return?* → One per employee (15).
2. *Where do rows with `dept_id` NULL go?* → Together in one partition.

TL;DR

`PARTITION BY` = `GROUP BY` without collapsing. Perfect for "share of group total" and "difference from group average".

8.3 ORDER BY inside OVER: running calculations and the default frame

What it is

Adding `ORDER BY` inside `OVER` does two things:

1. It defines the order that ranking and offset functions (`row_number`, `lag`, …) use.
2. For **aggregate** window functions it changes the default window from "the whole partition" to "from the start of the partition **up to the current row**" — so `sum` becomes a **running total**, `count` a

running count, `max` a running maximum.

```sql
-- code/02_order_and_default_frame.sql
SELECT store, day, amount,
       sum(amount) OVER (PARTITION BY store ORDER BY day) AS running_total,
       count(*)    OVER (PARTITION BY store ORDER BY day) AS days_so_far
FROM daily_sales
WHERE store = 'Mumbai'
ORDER BY day;
```

Output:

```
 store  |    day     | amount  | running_total | days_so_far
--------+------------+---------+---------------+-------------
 Mumbai | 2025-03-01 | 1200.00 |       1200.00 |           1
 Mumbai | 2025-03-02 |  900.00 |       2100.00 |           2
 Mumbai | 2025-03-03 | 1500.00 |       3600.00 |           3
 Mumbai | 2025-03-04 | 1500.00 |       5100.00 |           4
 Mumbai | 2025-03-05 |  700.00 |       5800.00 |           5
 Mumbai | 2025-03-06 | 1100.00 |       6900.00 |           6
 Mumbai | 2025-03-07 | 1800.00 |       8700.00 |           7
(7 rows)
```

The default frame and the tie trap

The exact default, when `ORDER BY` is present and no frame is written, is:

```sql
RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
```

`RANGE` works on *values*, not physical rows: "current row" means the current row **and all its peers** — rows with the same `ORDER BY` value. So when the ordering column has ties, tied rows are added together and all get the same "running" total:

```sql
SELECT first_name, salary,
       sum(salary) OVER (ORDER BY salary)                    AS default_range,
       sum(salary) OVER (ORDER BY salary
                         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS rows_frame,
       sum(salary) OVER (ORDER BY salary, emp_id)            AS unique_order
FROM employees
WHERE dept_id = 2
ORDER BY salary, emp_id;
```

Output:

```
 first_name |  salary   | default_range | rows_frame | unique_order
------------+-----------+---------------+------------+--------------
 Arjun      |  68000.00 |     136000.00 |   68000.00 |     68000.00
 Sara       |  68000.00 |     136000.00 |  136000.00 |    136000.00
 Priya      |  72000.00 |     208000.00 |  208000.00 |    208000.00
 Daniel     | 150000.00 |     358000.00 |  358000.00 |    358000.00
(4 rows)
```

- `default_range`: Arjun and Sara are peers, so both see 68 000 + 68 000.
- `rows_frame`: `ROWS` counts physical rows, so the total grows one row at a time — but which tied row comes "first" is **not defined**; next time Sara could get 68 000.
- `unique_order`: adding a tie-breaker (`emp_id`) makes the order unique, so there are no peers and every frame type gives the same, **deterministic** (identical on every run) result. This is the best fix.

`OVER (...)` contains	Default window for aggregates
nothing / only `PARTITION BY`	the whole partition
`ORDER BY`	`RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` (start → current row + its peers)

Common mistakes

- **Running totals that "jump"** on duplicate dates or amounts — the peer effect. Order by a unique key, or write `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`.
- **Confusing the window's `ORDER BY` with the query's `ORDER BY`**. The one inside `OVER` only controls the calculation; the output order is set by the final `ORDER BY` (without it, output order is not guaranteed).

Self-check

1. *Why can a running total show the same value on two rows?* → The default `RANGE` frame includes peers — rows with the same `ORDER BY` value.
2. *Two ways to avoid it?* → Make the `ORDER BY` unique (add a key) or use a `ROWS` frame (plus a tie-breaker for determinism).

TL;DR

`ORDER BY` in `OVER` turns aggregates into running aggregates. The default frame is `RANGE … CURRENT ROW`, which lumps ties together — add a unique tie-breaker.

8.4 Frames: ROWS, RANGE, GROUPS and EXCLUDE

What it is

The **frame** is the subset of the partition that an aggregate (and `first_value`/`last_value`/`nth_value`) looks at for the current row. It "slides" as the current row moves. Ranking functions and `lag`/`lead` ignore the frame.

```
{ ROWS | RANGE | GROUPS } BETWEEN frame_start AND frame_end [EXCLUDE ...]

frame_start / frame_end:
    UNBOUNDED PRECEDING      -- first row of the partition
    n PRECEDING              -- n rows / n units of value / n peer groups before
    CURRENT ROW
    n FOLLOWING
    UNBOUNDED FOLLOWING      -- last row of the partition
```

Short form: `ROWS 2 PRECEDING` means `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`.

Mode	`n PRECEDING` means	Peers (ties)	Needs
`ROWS`	*n physical rows* before	treated as separate rows	any `ORDER BY`
`RANGE`	rows whose **value** is within *n* of the current value (`n` can be a number or an `interval`)	always included together	exactly one `ORDER BY` column of a numeric/date/time type when an offset is used
`GROUPS` (PostgreSQL 11+)	*n peer groups* before (a group = rows with equal `ORDER BY` values)	included together	an `ORDER BY`

Frame diagrams

In the tables below, each line is one "current row" (**X**), and `x` marks the other rows inside its frame. The SQL uses `string_agg(...) OVER (frame)` to list the days in each frame — a handy trick to *see* a frame.

`ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` — a sliding window of up to 3 rows:

```sql
-- code/03_frames.sql
SELECT day, amount,
       string_agg(to_char(day, 'DD'), ',') OVER w AS frame_days,
       sum(amount) OVER w                         AS frame_sum
FROM daily_sales
WHERE store = 'Mumbai'
WINDOW w AS (ORDER BY day ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
ORDER BY day;
```

Output:

```
    day     | amount  | frame_days | frame_sum
------------+---------+------------+-----------
 2025-03-01 | 1200.00 | 01         |   1200.00
 2025-03-02 |  900.00 | 01,02      |   2100.00
 2025-03-03 | 1500.00 | 01,02,03   |   3600.00
 2025-03-04 | 1500.00 | 02,03,04   |   3900.00
 2025-03-05 |  700.00 | 03,04,05   |   3700.00
 2025-03-06 | 1100.00 | 04,05,06   |   3300.00
 2025-03-07 | 1800.00 | 05,06,07   |   3600.00
(7 rows)
```

current row	01	02	03	04	05	06	07
01	**X**	·	·	·	·	·	·
02	x	**X**	·	·	·	·	·
03	x	x	**X**	·	·	·	·
04	·	x	x	**X**	·	·	·
05	·	·	x	x	**X**	·	·
06	·	·	·	x	x	**X**	·
07	·	·	·	·	x	x	**X**

The first two frames are *shorter* — there are no earlier rows. (The `WINDOW w AS (...)` clause names a window so it can be reused; see 8.10.)

Centered: `ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING` — smoothing that looks both ways:

```sql
SELECT day, amount,
       string_agg(to_char(day, 'DD'), ',') OVER w AS frame_days,
       round(avg(amount) OVER w, 1)               AS centered_avg
FROM daily_sales
WHERE store = 'Mumbai'
WINDOW w AS (ORDER BY day ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING)
ORDER BY day;
```

Output:

```
    day     | amount  | frame_days | centered_avg
------------+---------+------------+--------------
 2025-03-01 | 1200.00 | 01,02      |       1050.0
 2025-03-02 |  900.00 | 01,02,03   |       1200.0
 ...
 2025-03-07 | 1800.00 | 06,07      |       1450.0
(7 rows)
```

`ROWS` vs `RANGE` when a day is missing. Pune has no row for 4 March. "The previous 2 rows" and "the previous 2 days" now differ:

```sql
SELECT day, amount,
       string_agg(to_char(day, 'DD'), ',') OVER (ORDER BY day
            ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)                    AS rows_2,
       string_agg(to_char(day, 'DD'), ',') OVER (ORDER BY day
            RANGE BETWEEN INTERVAL '2 days' PRECEDING AND CURRENT ROW)  AS range_2_days
FROM daily_sales
WHERE store = 'Pune'
ORDER BY day;
```

Output:

```
    day     | amount |  rows_2  | range_2_days
------------+--------+----------+--------------
 2025-03-01 | 400.00 | 01       | 01
 2025-03-02 | 650.00 | 01,02    | 01,02
 2025-03-03 | 500.00 | 01,02,03 | 01,02,03
 2025-03-05 | 800.00 | 02,03,05 | 03,05
 2025-03-06 | 650.00 | 03,05,06 | 05,06
 2025-03-07 | 300.00 | 05,06,07 | 05,06,07
(6 rows)
```

`ROWS 2 PRECEDING` (three rows, whatever their dates):

current row	01	02	03	05	06	07
01	**X**	·	·	·	·	·
02	x	**X**	·	·	·	·
03	x	x	**X**	·	·	·
05	·	x	x	**X**	·	·
06	·	·	x	x	**X**	·
07	·	·	·	x	x	**X**

`RANGE INTERVAL '2 days' PRECEDING` (dates from current − 2 days to current):

current row	01	02	03	05	06	07
01	**X**	·	·	·	·	·
02	x	**X**	·	·	·	·
03	x	x	**X**	·	·	·
05	·	·	x	**X**	·	·
06	·	·	·	x	**X**	·
07	·	·	·	x	x	**X**

For "average of the last 3 *calendar* days", `RANGE` with an interval is correct and `ROWS` silently reaches back to 2 March. (Alternatively, fill the missing days with `generate_series` first — chapter 7.)

`ROWS` vs `RANGE` vs `GROUPS` with ties. Ordering Pune by `amount` (two days have 650):

```sql
SELECT day, amount,
       string_agg(amount::int::text, ',') OVER (ORDER BY amount
            ROWS   BETWEEN 1 PRECEDING AND CURRENT ROW) AS rows_1,
       string_agg(amount::int::text, ',') OVER (ORDER BY amount
            RANGE  BETWEEN 150 PRECEDING AND CURRENT ROW) AS range_150,
       string_agg(amount::int::text, ',') OVER (ORDER BY amount
            GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW) AS groups_1
FROM daily_sales
WHERE store = 'Pune'
ORDER BY amount, day;
```

Output:

```
    day     | amount | rows_1  |  range_150  |  groups_1
------------+--------+---------+-------------+-------------
 2025-03-07 | 300.00 | 300     | 300         | 300
 2025-03-01 | 400.00 | 300,400 | 300,400     | 300,400
 2025-03-03 | 500.00 | 400,500 | 400,500     | 400,500
 2025-03-02 | 650.00 | 500,650 | 500,650,650 | 500,650,650
 2025-03-06 | 650.00 | 650,650 | 500,650,650 | 500,650,650
 2025-03-05 | 800.00 | 650,800 | 650,650,800 | 650,650,800
(6 rows)
```

- `ROWS 1 PRECEDING`: the previous physical row, so the two 650 rows get *different* frames.
- `RANGE 150 PRECEDING`: every row whose amount is between (current − 150) and current, **including all peers**. For 650: amounts 500…650 → 500, 650, 650.
- `GROUPS 1 PRECEDING`: the current peer group plus the one group before it. For 650: groups {500} and {650, 650}. For 800: groups {650, 650} and {800}.

`EXCLUDE` (PostgreSQL 11+) removes rows from the frame: `EXCLUDE CURRENT ROW`, `EXCLUDE GROUP` (current row and its peers), `EXCLUDE TIES` (peers but not the current row), `EXCLUDE NO OTHERS` (default). Example — each day compared with the average of all *other* days:

```sql
SELECT day, amount,
       round(avg(amount) OVER (ORDER BY day ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
                               EXCLUDE CURRENT ROW), 1) AS avg_of_other_days
FROM daily_sales
WHERE store = 'Mumbai'
ORDER BY day;
```

Output:

```
    day     | amount  | avg_of_other_days
------------+---------+-------------------
 2025-03-01 | 1200.00 |            1250.0
 2025-03-02 |  900.00 |            1300.0
 ...
 2025-03-07 | 1800.00 |            1150.0
(7 rows)
```

(Week total 8700; for 1 March: (8700 − 1200) / 6 = 1250.)

Choosing a frame

You want	Frame
Running total (deterministic)	`ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` + unique `ORDER BY`
Last *N* rows (e.g. last 7 records)	`ROWS BETWEEN N-1 PRECEDING AND CURRENT ROW`
Last *N* days of time, with gaps in data	`RANGE BETWEEN INTERVAL 'N-1 days' PRECEDING AND CURRENT ROW`
Whole partition (e.g. for `last_value`)	`ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` (or no `ORDER BY`)
Current value and the next distinct value(s)	`GROUPS BETWEEN CURRENT ROW AND 1 FOLLOWING`

Common mistakes

- **Writing `ROWS 6 PRECEDING` for "last 7 days" when dates can be missing** — use `RANGE` with an interval.
- **Forgetting the frame applies only to some functions.** `row_number`, `rank`, `lag`, `lead`, `ntile` ignore it.
- **Frame start after frame end** (e.g. `BETWEEN CURRENT ROW AND 1 PRECEDING`) — an error.
- **Porting `GROUPS` / `EXCLUDE` / `RANGE n PRECEDING` to SQL Server or MySQL** — not supported there (see the dialect table in 8.1).

In the interview

> **30-second answer:** "The frame says which rows around the current one an aggregate sees. `ROWS` counts physical rows, `RANGE` uses the ordering value — so ties are always included and you can say 'within 7 days' — and `GROUPS` counts groups of tied rows. The default with `ORDER BY` is `RANGE UNBOUNDED PRECEDING TO CURRENT ROW`, which is why running totals can repeat on ties."

- *Moving 7-day average with missing days?* → `avg(x) OVER (ORDER BY day RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW)`, or fill the calendar and use `ROWS 6 PRECEDING`.

Self-check

1. *Pune, current row 5 March: which days are in `ROWS 2 PRECEDING`? In `RANGE INTERVAL '2 days' PRECEDING`?* → 2, 3, 5 March; 3, 5 March.
2. *Which frame mode treats tied rows as separate?* → `ROWS`.
3. *What does `EXCLUDE CURRENT ROW` do?* → Removes the current row from its own frame.
4. *What is `ROWS 3 PRECEDING` short for?* → `ROWS BETWEEN 3 PRECEDING AND CURRENT ROW`.

TL;DR

Frame = which rows an aggregate sees. `ROWS` = physical rows, `RANGE` = value distance (ties included, intervals allowed), `GROUPS` = peer groups. `EXCLUDE` removes rows. Write the frame explicitly whenever it matters.

8.5 Ranking functions

What it is

Ranking functions give each row a number based on its position in the window's `ORDER BY`. They differ in **how they treat ties**.

Function	Returns	Ties
`row_number()`	1, 2, 3, … unique per row	tied rows get different numbers — **in no guaranteed order**
`rank()`	position with gaps ("Olympic" ranking)	same rank; next rank skips (1, 2, 2, 4)
`dense_rank()`	position without gaps	same rank; next rank is +1 (1, 2, 2, 3)
`percent_rank()`	(rank − 1) / (rows in partition − 1), from 0 to 1	same value
`cume_dist()`	(rows ordered before the current row + its peers) / rows in partition, from >0 to 1	same value
`ntile(n)`	bucket number 1…n, buckets as equal as possible	ties can be split across buckets

```sql
-- code/04_ranking.sql
SELECT first_name, salary,
       row_number()  OVER w AS row_number,
       rank()        OVER w AS rank,
       dense_rank()  OVER w AS dense_rank,
       round(percent_rank() OVER w :: numeric, 2) AS pct_rank,
       round(cume_dist()    OVER w :: numeric, 2) AS cume_dist,
       ntile(3)      OVER w AS ntile_3
FROM employees
WHERE dept_id IN (1, 2)
WINDOW w AS (ORDER BY salary DESC)
ORDER BY salary DESC, row_number;
```

Output:

```
 first_name |  salary   | row_number | rank | dense_rank | pct_rank | cume_dist | ntile_3
------------+-----------+------------+------+------------+----------+-----------+---------
 Vikram     | 185000.00 |          1 |    1 |          1 |     0.00 |      0.13 |       1
 Daniel     | 150000.00 |          2 |    2 |          2 |     0.14 |      0.25 |       1
 Neha       | 142000.00 |          3 |    3 |          3 |     0.29 |      0.38 |       1
 Rahul      |  98000.00 |          4 |    4 |          4 |     0.43 |      0.63 |       2
 Meera      |  98000.00 |          5 |    4 |          4 |     0.43 |      0.63 |       2
 Priya      |  72000.00 |          6 |    6 |          5 |     0.71 |      0.75 |       2
 Arjun      |  68000.00 |          7 |    7 |          6 |     0.86 |      1.00 |       3
 Sara       |  68000.00 |          8 |    7 |          6 |     0.86 |      1.00 |       3
(8 rows)
```

Reading the ties:

- Rahul and Meera tie at 98 000: `rank` 4 and 4, then Priya gets **6** (position 5 is "used up"); `dense_rank` gives 4, 4, then **5**.
- `row_number` gave Rahul 4 and Meera 5 — but nothing in `ORDER BY salary DESC` says so; it could flip. Add a tie-breaker (`ORDER BY salary DESC, emp_id`) whenever you use `row_number`.
- `percent_rank` for Priya: (6 − 1) / (8 − 1) = 0.71. `cume_dist` for Rahul/Meera: 5 of 8 rows have salary ≥ 98 000 in this descending order → 0.63.
- `ntile(3)` over 8 rows: 8 / 3 = 2 remainder 2, so the first two buckets get 3 rows and the last gets 2.

Ranking within groups — add `PARTITION BY`:

```sql
SELECT dept_id, first_name, salary,
       dense_rank() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rank_in_dept
FROM employees
WHERE dept_id IS NOT NULL
ORDER BY dept_id, rank_in_dept, first_name;
```

Output:

```
 dept_id | first_name |  salary   | rank_in_dept
---------+------------+-----------+--------------
       1 | Vikram     | 185000.00 |            1
       1 | Neha       | 142000.00 |            2
       1 | Meera      |  98000.00 |            3
       1 | Rahul      |  98000.00 |            3
 ...
       5 | Li         |  64000.00 |            1
(13 rows)
```

`ntile` **for quartiles.** Bucket everyone into four salary quartiles (the four equal-sized quarters of the sorted values), then summarise each bucket (window in a subquery, `GROUP BY` outside):

```sql
SELECT quartile, count(*) AS people, min(salary), max(salary)
FROM (SELECT salary, ntile(4) OVER (ORDER BY salary) AS quartile FROM employees) t
GROUP BY quartile
ORDER BY quartile;
```

Output:

```
 quartile | people |    min    |    max
----------+--------+-----------+-----------
        1 |      4 |  30000.00 |  68000.00
        2 |      4 |  68000.00 |  98000.00
        3 |      4 |  98000.00 | 142000.00
        4 |      3 | 150000.00 | 250000.00
(4 rows)
```

Note that 68 000 and 98 000 appear in two buckets each: `ntile` fills buckets by row count and **splits ties**. If tied values must stay together, compute buckets from `percent_rank` or `cume_dist` instead (or

use `percentile_cont`, an ordered-set aggregate — an aggregate that reads its input in sorted order, like a median — for exact quartile boundaries — chapter 4).

Which ranking function? (the question interviewers love)

Question	Use
"Exactly one row per …" / pagination (splitting results into pages) / dedup	`row_number()` with a unique tie-breaker
"Top 3 places, ties share a place, next place skips" (sports)	`rank()`
"The 2nd highest *distinct* salary" / "top 3 salary levels"	`dense_rank()`
"Top 10 %"	`percent_rank() <= 0.1` or `cume_dist() <= 0.1` (depending on how ties should count)
"Split into 4 equal groups"	`ntile(4)`

Nth highest value — the classic: `dense_rank` makes ties count once.

```
-- 2nd highest distinct salary (NULL if there isn't one)
SELECT max(salary) FROM (
    SELECT salary, dense_rank() OVER (ORDER BY salary DESC) AS dr FROM employees
) t WHERE dr = 2;
```

Output:

```
    max
-----------
 185000.00
(1 row)
```

Asha, the CEO, earns 250 000, so Vikram's 185 000 is the second-highest distinct value. Wrapping in `max()` returns one row even when several people share that salary, and `NULL` rather than no rows when there is no 2nd value.

Self-check

1. *Salaries 100, 90, 90, 80: rank of 80? dense_rank?* → 4; 3.
2. *Why add `emp_id` to `ORDER BY` for `row_number`?* → To make the numbering deterministic among ties.
3. *`ntile(4)` over 10 rows — bucket sizes?* → 3, 3, 2, 2.
4. *Which function answers "second highest distinct salary"?* → `dense_rank() = 2`.

TL;DR

`row_number` (unique, arbitrary among ties), `rank` (ties share, gaps after), `dense_rank` (ties share, no gaps), `ntile(n)` (n near-equal buckets), `percent_rank`/`cume_dist` (relative position 0–1). Always think about ties.

8.6 Value (offset) functions: LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE

What it is

Value functions return a value **from another row** of the window.

Function	Returns	Uses the frame?
`lag(expr [, offset [, default]])`	`expr` from `offset` rows **before** (default 1)	no — relative to the partition's order
`lead(expr [, offset [, default]])`	`expr` from `offset` rows **after**	no
`first_value(expr)`	`expr` from the frame's first row	yes
`last_value(expr)`	`expr` from the frame's last row	yes
`nth_value(expr, n)`	`expr` from the frame's *n*-th row (1-based), NULL if the frame is shorter	yes

When there is no such row (the first row has no "previous"), `lag`/`lead` return `NULL`, or the `default` you pass.

Day-over-day change:

```sql
-- code/05_value_functions.sql
SELECT store, day, amount,
       lag(amount)  OVER w          AS prev_day,
       amount - lag(amount) OVER w  AS change,
       lead(amount) OVER w          AS next_day,
       lag(amount, 2, 0) OVER w     AS two_days_ago_or_0
FROM daily_sales
WHERE day <= '2025-03-04'
WINDOW w AS (PARTITION BY store ORDER BY day)
ORDER BY store, day;
```

Output:

```
 store  |    day     | amount  | prev_day | change  | next_day | two_days_ago_or_0
--------+------------+---------+----------+---------+----------+-------------------
 Mumbai | 2025-03-01 | 1200.00 |   (null) |  (null) |   900.00 |                 0
 Mumbai | 2025-03-02 |  900.00 |  1200.00 | -300.00 |  1500.00 |                 0
 ...
 Mumbai | 2025-03-04 | 1500.00 |  1500.00 |    0.00 |   (null) |            900.00
 Pune   | 2025-03-01 |  400.00 |   (null) |  (null) |   650.00 |                 0
 Pune   | 2025-03-02 |  650.00 |   400.00 |  250.00 |   500.00 |                 0
 Pune   | 2025-03-03 |  500.00 |   650.00 | -150.00 |   (null) |            400.00
(7 rows)
```

`PARTITION BY store` stops `lag` from reaching into the other store's rows: Pune's first day has no previous day even though Mumbai's last row came before it. The `WHERE` runs before the window, so `lead` on 4 March sees nothing (5 March was filtered out).

Note that `lag` means "previous **row**", not "previous **day**": in Pune, the row before 5 March is 3 March. If you need the previous *calendar* day, fill gaps first or check the date difference (8.9).

The `last_value` trap. With `ORDER BY` in the window, the default frame ends at the current row, so `last_value` just returns the current row:

```sql
SELECT day, amount,
       first_value(amount) OVER (ORDER BY day)    AS first_amt,
       last_value(amount)  OVER (ORDER BY day)    AS last_default_frame,
       last_value(amount)  OVER (ORDER BY day
            ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS last_whole_partition,
       nth_value(amount, 2) OVER (ORDER BY day
            ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS second_amt
FROM daily_sales
WHERE store = 'Pune'
ORDER BY day;
```

Output:

```
    day     | amount | first_amt | last_default_frame | last_whole_partition | second_amt
------------+--------+-----------+--------------------+----------------------+------------
 2025-03-01 | 400.00 |    400.00 |             400.00 |               300.00 |     650.00
 2025-03-02 | 650.00 |    400.00 |             650.00 |               300.00 |     650.00
 2025-03-03 | 500.00 |    400.00 |             500.00 |               300.00 |     650.00
 ...
 2025-03-07 | 300.00 |    400.00 |             300.00 |               300.00 |     650.00
(6 rows)
```

Fix: extend the frame to `UNBOUNDED FOLLOWING`, or use `first_value` with the order reversed (`first_value(amount) OVER (ORDER BY day DESC)`), which needs no frame.

`first_value(first_name) OVER (PARTITION BY dept_id ORDER BY salary DESC)` puts each department's top earner beside every employee — handy for "gap to the leader" columns (the file shows it).

Time between events — days between a customer's consecutive orders (a standard retention metric, i.e. a measure of how often customers come back):

```sql
SELECT customer_id, order_id, placed_at::date AS placed,
       placed_at::date - lag(placed_at::date) OVER (PARTITION BY customer_id ORDER BY placed_at) AS
days_since_prev
FROM orders
WHERE customer_id IN (1, 2, 5)
ORDER BY customer_id, placed_at;
```

Output:

```
 customer_id | order_id |   placed   | days_since_prev
-------------+----------+------------+-----------------
           1 |      101 | 2025-01-03 |          (null)
           1 |      104 | 2025-02-11 |              39
           1 |      113 | 2025-06-02 |             111
           2 |      102 | 2025-01-17 |          (null)
           2 |      108 | 2025-04-01 |              74
 ...
(7 rows)
```

Subtracting two `date`s gives an integer number of days in PostgreSQL.

Dialect notes

- `IGNORE NULLS` (e.g. `lag(x) IGNORE NULLS OVER (...)`, "the last non-null value") is in the SQL standard and supported by Oracle, SQL Server 2022, BigQuery and Snowflake (two cloud data-warehouse products). PostgreSQL gains it in **version 19** (in beta as of September 2026); PostgreSQL 14–18 and MySQL lack it. There, emulate it with a running `count(x)` as a group key and `first_value` within each group, or a correlated subquery.
- `nth_value(...) FROM LAST` (counting from the end of the frame) exists in Oracle and a few warehouses, not in PostgreSQL, MySQL or SQL Server.

Common mistakes

- `last_value` without a full frame; `lag` without `PARTITION BY` (crosses groups); `lag` treated as "previous period" when periods can be missing.
- The first row's `amount - lag(amount)` is `NULL`; pass a `default` or use `coalesce` if you want 0.

In the interview

30-second answer: "`LAG` and `LEAD` read a column from a previous or next row in the window's order, per partition, which replaces a self-join on 'row n−1'. `FIRST_VALUE`, `LAST_VALUE` and `NTH_VALUE` read from positions in the frame — and because the default frame ends at the current row, `LAST_VALUE` needs `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`."

- *Year-over-year growth per product?* → `lag(revenue) OVER (PARTITION BY product ORDER BY year)`, then `(revenue - prev) / prev`.

- *How did people do this before window functions?* → A self-join on `t2.day = t1.day - 1` (breaks with gaps) or a correlated subquery `(SELECT ... WHERE t2.day < t1.day ORDER BY day DESC LIMIT 1)`.

Self-check

1. *What does `lag(x, 1, 0)` return on the first row?* → 0.
2. *Why does `last_value(x) OVER (ORDER BY d)` equal `x`?* → The default frame ends at the current row (plus peers).
3. *Which of these ignore the frame: `lag`, `first_value`, `lead`, `nth_value`?* → `lag` and `lead`.

TL;DR

`lag` / `lead` = previous/next row per partition (with optional offset and default).
`first_value` / `last_value` / `nth_value` read the frame — give `last_value` a full frame.

8.7 Running totals, moving averages and growth

What it is

The everyday analytics calculations are just aggregate window functions with the right frame:

Metric	Window
Running total	`sum(x) OVER (PARTITION BY g ORDER BY t ROWS UNBOUNDED PRECEDING)`
Moving average of last 3 rows	`avg(x) OVER (PARTITION BY g ORDER BY t ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)`
Cumulative share	running total / `sum(x) OVER (PARTITION BY g)`
Share of total	`x / sum(x) OVER (PARTITION BY g)`
Growth vs previous period	`x / lag(x) OVER (ORDER BY t) - 1`

```sql
-- code/06_running_and_moving.sql
SELECT store, day, amount,
       sum(amount) OVER (PARTITION BY store ORDER BY day
                         ROWS UNBOUNDED PRECEDING)                        AS running_total,
       round(avg(amount) OVER (PARTITION BY store ORDER BY day
                         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 1)     AS moving_avg_3,
       count(*) OVER (PARTITION BY store ORDER BY day
                         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)         AS rows_in_avg,
       round(100.0 * sum(amount) OVER (PARTITION BY store ORDER BY day ROWS UNBOUNDED PRECEDING)
                   / sum(amount) OVER (PARTITION BY store), 1)             AS pct_of_week_so_far
FROM daily_sales
ORDER BY store, day;
```

Output:

```
 store  |    day     | amount  | running_total | moving_avg_3 | rows_in_avg | pct_of_week_so_far
--------+------------+---------+---------------+--------------+-------------+--------------------
 Mumbai | 2025-03-01 | 1200.00 |       1200.00 |       1200.0 |           1 |               13.8
 Mumbai | 2025-03-02 |  900.00 |       2100.00 |       1050.0 |           2 |               24.1
 Mumbai | 2025-03-03 | 1500.00 |       3600.00 |       1200.0 |           3 |               41.4
 ...
 Mumbai | 2025-03-07 | 1800.00 |       8700.00 |       1200.0 |           3 |              100.0
 ...
 Pune   | 2025-03-03 |  500.00 |       1550.00 |        516.7 |           3 |               47.0
 Pune   | 2025-03-05 |  800.00 |       2350.00 |        650.0 |           3 |               71.2
 ...
 Pune   | 2025-03-07 |  300.00 |       3300.00 |        583.3 |           3 |              100.0
(13 rows)
```

Two things to notice:

1. The first two moving averages use only 1 and 2 rows (`rows_in_avg`) — a "warm-up" period. Averages over fewer points than intended can mislead on a chart.
2. Pune's 5 March average (650.0) uses 2, 3 and 5 March because the frame is `ROWS` — see 8.4 for the `RANGE` alternative.

To hide warm-up rows, show the average only when the frame is full: `CASE WHEN count(*) OVER w = 3 THEN avg(amount) OVER w END` (in `code/06_running_and_moving.sql`).

Windows on top of GROUP BY

Window functions run **after** `GROUP BY`, so they can work on the grouped rows. That's how you get "monthly revenue and the running total of monthly revenue" in one query. The inner `sum` is the normal aggregate (per month); the outer `sum(...) OVER` adds up those monthly sums:

```
SELECT date_trunc('month', o.placed_at)::date AS month,
       sum(oi.quantity * oi.unit_price * (1 - oi.discount))::numeric(10,2) AS revenue,
       sum(sum(oi.quantity * oi.unit_price * (1 - oi.discount)))
           OVER (ORDER BY date_trunc('month', o.placed_at)::date)::numeric(10,2) AS running_revenue
FROM orders o JOIN order_items oi USING (order_id)
WHERE o.status <> 'cancelled'
GROUP BY 1
ORDER BY 1;
```

Output:

```
   month    | revenue | running_revenue
------------+---------+-----------------
 2025-01-01 | 1871.90 |         1871.90
 2025-02-01 |  811.49 |         2683.39
 ...
 2025-06-01 | 1516.60 |         7403.62
(6 rows)
```

`sum(sum(...))` looks odd but is correct. It's often clearer to aggregate in a CTE (Common Table Expression — a named subquery written with `WITH`, chapter 7) first and apply windows in the outer query — as here for **month-over-month growth**:

```
WITH monthly AS (
    SELECT date_trunc('month', o.placed_at)::date AS month,
           sum(oi.quantity * oi.unit_price * (1 - oi.discount)) AS revenue
    FROM orders o JOIN order_items oi USING (order_id)
    WHERE o.status <> 'cancelled'
    GROUP BY 1
)
SELECT month, round(revenue, 2) AS revenue,
       round(lag(revenue) OVER (ORDER BY month), 2) AS prev_month,
       round(100.0 * (revenue / lag(revenue) OVER (ORDER BY month) - 1), 1) AS mom_growth_pct
FROM monthly
ORDER BY month;
```

Output:

```
   month    | revenue | prev_month | mom_growth_pct
------------+---------+------------+----------------
 2025-01-01 | 1871.90 |     (null) |         (null)
 2025-02-01 |  811.49 |    1871.90 |          -56.6
 2025-03-01 |  804.95 |     811.49 |           -0.8
 ...
 2025-06-01 | 1516.60 |     249.93 |          506.8
(6 rows)
```

If a month could have *no* orders, generate the months first (chapter 7) — otherwise `lag` would compare with two months back. Guard divisions with `NULLIF(prev, 0)` if the previous value can be zero.

Real-world examples

Account balance after each transaction (running `sum` ordered by time and id), 7-day moving average of daily active users, stock level as a running sum of `+received -shipped`.

TL;DR

Running total = `sum` + `ROWS UNBOUNDED PRECEDING`; moving average = `avg` + `ROWS n PRECEDING` (watch warm-up rows and gaps); growth = value / `lag(value)` − 1. Windows run after `GROUP BY`, so `sum(sum(x)) OVER (...)` works — a CTE is often clearer.

8.8 Top-N per group and de-duplication

The problem

"Top 2 earners in each department", "latest order per customer", "best-selling product per category" — these **top-N-per-group** questions are the most common window-function interview problem. `LIMIT` can't do it (it limits the whole result, not each group), and `GROUP BY` returns only the aggregated value, not the rest of the row.

Why you need a subquery or CTE

Window functions are computed *after* `WHERE`, so you cannot filter on them in the same query level:

```sql
-- code/07_top_n_per_group.sql
SELECT first_name, salary FROM employees
WHERE row_number() OVER (ORDER BY salary DESC) <= 3;
```

Output:

```
ERROR:  window functions are not allowed in WHERE
LINE 2: WHERE row_number() OVER (ORDER BY salary DESC) <= 3;
              ^
```

(They're not allowed in `GROUP BY` or `HAVING` either.) The pattern is always: **compute the rank in a CTE or subquery, filter in the outer query.**

How it works

1. In a CTE, add `row_number()` / `rank()` / `dense_rank()` `OVER (PARTITION BY group ORDER BY measure DESC)`.
2. In the outer query, `WHERE rank_col <= N`.
3. Pick the ranking function according to how ties should be treated.

```sql
WITH ranked AS (
    SELECT d.name AS dept, e.first_name, e.salary,
           row_number() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC, e.emp_id) AS rn,
           rank()       OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC)           AS rnk,
           dense_rank() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC)           AS drnk
    FROM employees e JOIN departments d USING (dept_id)
)
SELECT dept, first_name, salary, rn, rnk, drnk
FROM ranked
WHERE dept IN ('Engineering', 'Sales') AND rnk <= 3
ORDER BY dept, rn;
```

Output:

```
    dept     | first_name |  salary   | rn | rnk | drnk
-------------+------------+-----------+----+-----+------
 Engineering | Vikram     | 185000.00 |  1 |   1 |    1
 ...
 Engineering | Rahul      |  98000.00 |  3 |   3 |    3
 Engineering | Meera      |  98000.00 |  4 |   3 |    3
 Sales       | Daniel     | 150000.00 |  1 |   1 |    1
 Sales       | Priya      |  72000.00 |  2 |   2 |    2
 Sales       | Arjun      |  68000.00 |  3 |   3 |    3
 Sales       | Sara       |  68000.00 |  4 |   3 |    3
(8 rows)
```

The three filters give different answers when there are ties — here for Sales:

```sql
-- ranked = Sales only; then:
SELECT 'row_number <= 3' AS rule, string_agg(first_name, ', ' ORDER BY rn) AS kept FROM ranked WHERE rn <= 3
UNION ALL
SELECT 'rank <= 3',       string_agg(first_name, ', ' ORDER BY rn) FROM ranked WHERE rnk <= 3
UNION ALL
SELECT 'dense_rank <= 2', string_agg(first_name, ', ' ORDER BY rn) FROM ranked WHERE drnk <= 2;
```

Output:

```
      rule       |            kept
-----------------+----------------------------
 row_number <= 3 | Daniel, Priya, Arjun
 rank <= 3       | Daniel, Priya, Arjun, Sara
 dense_rank <= 2 | Daniel, Priya
(3 rows)
```

Filter	Meaning	Rows per group
`row_number() <= N`	exactly N rows; ties broken by your tie-breaker	exactly N (or fewer)
`rank() <= N`	everyone whose position is within the top N; ties included	can exceed N
`dense_rank() <= N`	everyone having one of the top N *distinct* values	can far exceed N

Ask in an interview: "If two people tie for 3rd, should both be returned?"

Latest row per key (de-duplication). Same pattern with `N = 1`, ordered newest first:

```sql
SELECT customer_id, order_id, placed_at, status
FROM (
    SELECT o.*, row_number() OVER (PARTITION BY customer_id ORDER BY placed_at DESC) AS rn
    FROM orders o
) t
WHERE rn = 1
ORDER BY customer_id;
```

Output:

```
 customer_id | order_id |      placed_at      |  status
-------------+----------+---------------------+-----------
           1 |      113 | 2025-06-02 10:30:00 | pending
           2 |      108 | 2025-04-01 12:00:00 | delivered
           3 |      111 | 2025-05-06 07:55:00 | cancelled
 ...
           8 |      109 | 2025-04-09 16:45:00 | delivered
(8 rows)
```

Find duplicates to delete — number the rows within each duplicate group, and everything with `rn > 1` is an extra copy. The duplicate Anita Shah (chapter 1's messy email) is found after normalising the email:

```sql
WITH dups AS (
    SELECT customer_id,
           row_number() OVER (PARTITION BY lower(trim(email)) ORDER BY customer_id) AS rn
    FROM customers
    WHERE email IS NOT NULL
)
SELECT customer_id AS would_delete FROM dups WHERE rn > 1;
```

Output:

```
 would_delete
--------------
            8
(1 row)
```

To actually delete: `DELETE FROM customers WHERE customer_id IN (SELECT customer_id FROM dups WHERE rn > 1)` (after re-pointing that customer's orders — chapter 9 covers `DELETE` and foreign keys — columns whose values must match a key in another table).

Alternatives

Approach	Top-1	Top-N	Notes
Window + filter (above)	yes	yes	portable, flexible about ties
`DISTINCT ON (group) ... ORDER BY group, measure DESC`	yes	no	PostgreSQL only, concise
`LATERAL (... ORDER BY ... LIMIT N)` per group (ch. 6)	yes	yes	fastest when an index (sorted lookup structure) on (group, measure) exists and groups are few
Correlated subquery `WHERE salary = (SELECT max(...) ...)`	yes (with ties)	awkward	classic pre-window approach
`QUALIFY rn <= N`	yes	yes	filter on windows without a subquery — Snowflake, BigQuery, DuckDB, Teradata; **not** PostgreSQL / MySQL / SQL Server

For top-1, `SELECT DISTINCT ON (dept_id) dept_id, first_name, salary FROM employees WHERE dept_id IS NOT NULL ORDER BY dept_id, salary DESC, emp_id` returns each department's best-paid person in one short query (the file runs it).

In the interview

30-second answer: "Rank rows inside each group with a window function in a CTE — `PARTITION BY` the group, `ORDER BY` the measure descending — then filter `rank <= N` in the outer query, because window functions can't appear in `WHERE`. I'd choose `row_number`, `rank` or `dense_rank` depending on whether ties should be included, and add a tie-breaker for determinism."

- *Top 3 salaries per department with ties?* → `dense_rank() <= 3` (three distinct salary levels) or `rank() <= 3` (three places).
- *How to make it fast on a big table?* → Index `(dept_id, salary DESC)`; PostgreSQL can then read each partition pre-sorted; for few groups a `LATERAL ... LIMIT N` uses the index directly.

Self-check

1. *Why can't you write `WHERE rank() OVER (...) = 1`?* → Window functions are evaluated after `WHERE`.
2. *Top 2 per group, ties allowed to exceed 2 — which function?* → `rank()` (or `dense_rank()` for distinct values).
3. *How do you keep only the newest row per `user_id`?* → `row_number() OVER (PARTITION BY user_id ORDER BY created_at DESC, id DESC) = 1` in a subquery.

TL;DR

Rank in a CTE, filter outside. `row_number` = exactly N, `rank` = N places with ties, `dense_rank` = N distinct values. `rn = 1` newest-first is the standard dedup. PostgreSQL shortcut for top-1: `DISTINCT ON`.

8.9 Gaps and islands

What it is

An **island** is a run of consecutive values (consecutive days a user logged in, consecutive seat numbers that are free, consecutive readings above a threshold). A **gap** is a missing stretch between islands. "Longest login streak", "find missing invoice numbers" and "group events into sessions" are all gaps-and-islands problems.

How it works — the row-number trick

For a run of consecutive dates, the date goes up by 1 each row and so does `row_number()`. Their **difference is therefore constant within a run** and changes when a day is skipped. That constant identifies the island.

1. Number the rows per user in date order: `rn = row_number() OVER (PARTITION BY user_id ORDER BY login_date)`.
2. Compute `grp = login_date - rn` (a date minus an integer is a date).
3. `GROUP BY user_id, grp`: each group is one island; `min` / `max` / `count` describe it.

```sql
-- code/08_gaps_and_islands.sql
SELECT user_id, login_date,
       row_number() OVER (PARTITION BY user_id ORDER BY login_date) AS rn,
       login_date - row_number() OVER (PARTITION BY user_id ORDER BY login_date)::int AS grp
FROM logins
WHERE user_id = 1
ORDER BY login_date;
```

Output:

```
 user_id | login_date | rn |    grp
---------+------------+----+------------
       1 | 2025-05-01 |  1 | 2025-04-30
       1 | 2025-05-02 |  2 | 2025-04-30
       1 | 2025-05-03 |  3 | 2025-04-30
       1 | 2025-05-06 |  4 | 2025-05-02
       1 | 2025-05-07 |  5 | 2025-05-02
       1 | 2025-05-10 |  6 | 2025-05-04
(6 rows)
```

The value of `grp` has no meaning of its own — it's just a label shared by the rows of one run. (`row_number()` returns `bigint`; the `::int` cast is needed because PostgreSQL defines `date - integer`, not `date - bigint`.)

Each island (streak):

```sql
WITH marked AS (
    SELECT user_id, login_date,
           login_date - row_number() OVER (PARTITION BY user_id ORDER BY login_date)::int AS grp
    FROM logins
)
SELECT user_id, min(login_date) AS streak_start, max(login_date) AS streak_end,
       count(*) AS days
FROM marked
GROUP BY user_id, grp
ORDER BY user_id, streak_start;
```

Output:

```
 user_id | streak_start | streak_end | days
---------+--------------+------------+------
       1 | 2025-05-01   | 2025-05-03 |    3
       1 | 2025-05-06   | 2025-05-07 |    2
       1 | 2025-05-10   | 2025-05-10 |    1
       2 | 2025-05-02   | 2025-05-05 |    4
       2 | 2025-05-09   | 2025-05-10 |    2
(5 rows)
```

Longest streak per user — one more step (`DISTINCT ON` keeps the top row per user):

```sql
WITH marked AS (...same as above...),
streaks AS (
    SELECT user_id, grp, count(*) AS days, min(login_date) AS from_day
    FROM marked GROUP BY user_id, grp
)
SELECT DISTINCT ON (user_id) user_id, days AS longest_streak, from_day
FROM streaks
ORDER BY user_id, days DESC, from_day;
```

Output:

```
 user_id | longest_streak |  from_day
---------+----------------+------------
       1 |              3 | 2025-05-01
       2 |              4 | 2025-05-02
(2 rows)
```

If the data can contain the same date twice for a user, de-duplicate first (or use `dense_rank` instead of `row_number`), otherwise the trick breaks.

Gaps with `lead`. Compare each row with the next one; a difference greater than 1 is a gap:

```sql
SELECT user_id,
       login_date + 1              AS gap_start,
       next_login - 1              AS gap_end,
       next_login - login_date - 1 AS missing_days
FROM (
    SELECT user_id, login_date,
           lead(login_date) OVER (PARTITION BY user_id ORDER BY login_date) AS next_login
    FROM logins
) t
WHERE next_login - login_date > 1
ORDER BY user_id, gap_start;
```

Output:

```
 user_id | gap_start  |  gap_end   | missing_days
---------+------------+------------+--------------
       1 | 2025-05-04 | 2025-05-05 |            2
       1 | 2025-05-08 | 2025-05-09 |            2
       2 | 2025-05-06 | 2025-05-08 |            3
(3 rows)
```

The same query with integer ids finds missing invoice or ticket numbers.

The general method: flag and running sum

The row-number trick needs "consecutive" to mean "+1". For any other rule — events less than 30 minutes apart, temperature above a threshold, same status as the previous row — use two steps:

1. **Flag** the first row of each island: `CASE WHEN <this row does NOT continue the previous one> THEN 1 ELSE 0 END`, using `lag`.
2. **Running sum** of the flags: `sum(flag) OVER (ORDER BY ...)` numbers the islands 1, 2, 3, …

Here: **sessionization** — a new session starts when more than 30 minutes have passed since the previous event:

```sql
WITH ev(user_id, at) AS (VALUES
    (7, TIMESTAMP '2025-05-01 09:00'), (7, '2025-05-01 09:10'), (7, '2025-05-01 09:25'),
    (7, '2025-05-01 11:00'), (7, '2025-05-01 11:05'),
    (7, '2025-05-01 15:30')
), flagged AS (
    SELECT user_id, at,
           CASE WHEN at - lag(at) OVER (PARTITION BY user_id ORDER BY at) <= INTERVAL '30 minutes'
                THEN 0 ELSE 1 END AS new_session
    FROM ev
)
SELECT user_id, at, new_session,
       sum(new_session) OVER (PARTITION BY user_id ORDER BY at) AS session_no
FROM flagged
ORDER BY at;
```

Output:

```
 user_id |         at          | new_session | session_no
---------+---------------------+-------------+------------
       7 | 2025-05-01 09:00:00 |           1 |          1
       7 | 2025-05-01 09:10:00 |           0 |          1
       7 | 2025-05-01 09:25:00 |           0 |          1
       7 | 2025-05-01 11:00:00 |           1 |          2
       7 | 2025-05-01 11:05:00 |           0 |          2
       7 | 2025-05-01 15:30:00 |           1 |          3
(6 rows)
```

The first event has no previous one: `lag` is `NULL`, the comparison is `NULL` (not true), so the `CASE` falls to `ELSE 1` — a new session, as intended. Group by `(user_id, session_no)` to get session start, end and length. (`VALUES` as a CTE creates inline sample rows; the first row's `TIMESTAMP` literal sets the column type.)

Real-world examples

App streaks and contribution graphs, web-analytics sessions (30 minutes of inactivity is a common cut-off), periods when CPU (the processor) load stayed above 90 %, missing cheque or invoice numbers.

In the interview

30-second answer: "For consecutive dates or integers, subtract `row_number()` from the value; the difference is constant within each run, so grouping by it gives the islands with `min`, `max`, `count`. For general rules, flag each row that starts a new island using `lag`, then a running `sum` of the flags gives an island id. Gaps come from `lead(value) - value > 1`."

- *Users who logged in 3+ consecutive days?* → Islands query, then `HAVING count(*) >= 3`, select distinct users.
- *What if a user logs in twice on the same day?* → De-duplicate to one row per day first, or use `dense_rank`.

Self-check

1. *Why is `login_date - row_number()` constant inside a streak?* → Both increase by exactly 1 per row while the days are consecutive.
2. *What does the running sum of "new island" flags give?* → A sequential island number per row.
3. *How do you list missing numbers between consecutive ids?* → `lead(id) - id > 1` → gap from `id + 1` to `lead(id) - 1`.

TL;DR

Islands: `value - row_number()` is a run label → `GROUP BY` it. General rule: flag starts with `lag`, running `sum` of flags = island id. Gaps: `lead(v) - v > 1`.

8.10 Named windows, FILTER, evaluation order and performance

The WINDOW clause

When several functions share a window, define it once with `WINDOW name AS (...)` (after `HAVING`, before `ORDER BY`) and write `OVER name`. A window can also be based on another: `OVER (w ROWS 2 PRECEDING)` adds a frame to named window `w` (the base window must not have its own frame).

```sql
-- code/09_named_windows_filter_plan.sql
SELECT store, day, amount,
       sum(amount) OVER w_run    AS running_total,
       max(amount) OVER w_run    AS best_so_far,
       rank()      OVER w_amount AS amount_rank
FROM daily_sales
WHERE store = 'Pune'
WINDOW w_run    AS (PARTITION BY store ORDER BY day),
       w_amount AS (PARTITION BY store ORDER BY amount DESC)
ORDER BY day;
```

Output:

```
 store |    day     | amount | running_total | best_so_far | amount_rank
-------+------------+--------+---------------+-------------+-------------
 Pune  | 2025-03-01 | 400.00 |        400.00 |      400.00 |           5
 Pune  | 2025-03-02 | 650.00 |       1050.00 |      650.00 |           2
 Pune  | 2025-03-03 | 500.00 |       1550.00 |      650.00 |           4
 Pune  | 2025-03-05 | 800.00 |       2350.00 |      800.00 |           1
 Pune  | 2025-03-06 | 650.00 |       3000.00 |      800.00 |           2
 Pune  | 2025-03-07 | 300.00 |       3300.00 |      800.00 |           6
(6 rows)
```

`max(...) OVER w_run` is a running maximum ("best day so far"). The `WINDOW` clause is supported by PostgreSQL, MySQL 8, SQLite and SQL Server 2022 (not earlier SQL Server).

FILTER on window aggregates

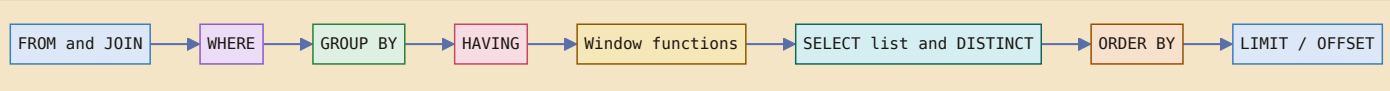
The `FILTER (WHERE ...)` clause from chapter 4 also works on aggregate window functions (PostgreSQL and SQLite; elsewhere use `count(CASE WHEN cond THEN 1 END) OVER (...)`):

```sql
SELECT day, amount,
       count(*) FILTER (WHERE amount >= 1500) OVER (ORDER BY day) AS big_days_so_far
FROM daily_sales
WHERE store = 'Mumbai'
ORDER BY day;
```

Output:

```
    day     | amount  | big_days_so_far
------------+---------+-----------------
 2025-03-01 | 1200.00 |               0
 2025-03-02 |  900.00 |               0
 2025-03-03 | 1500.00 |               1
 2025-03-04 | 1500.00 |               2
 2025-03-05 |  700.00 |               2
 2025-03-06 | 1100.00 |               2
 2025-03-07 | 1800.00 |               3
(7 rows)
```

Where windows fit in the logical order



Consequences:

- `WHERE` **changes the window.** `rank() OVER (ORDER BY salary DESC)` with `WHERE dept_id = 1` ranks only Engineering (Vikram 1, Neha 2, Rahul and Meera 3). For company-wide ranks shown only for Engineering, rank in a subquery **without** the filter and filter outside.
- Windows **can't** be used in `WHERE`, `GROUP BY` or `HAVING` (they don't exist yet) → wrap in a subquery/CTE (8.8).
- Windows **can** be used in `ORDER BY` (`ORDER BY rank() OVER (...)`) and can take aggregates as arguments in grouped queries (8.7).
- `DISTINCT` runs after windows, so `SELECT DISTINCT dept_id, count(*) OVER (PARTITION BY dept_id)` works (one row per department).
- `LIMIT` runs last, so `row_number() ... LIMIT 5` numbers *all* rows first.

Performance

PostgreSQL evaluates windows with a **WindowAgg** node (one step in the execution plan that `EXPLAIN` prints) that reads rows sorted by the `PARTITION BY` + `ORDER BY` keys:

```
EXPLAIN (COSTS OFF)
SELECT store, day, sum(amount) OVER (PARTITION BY store ORDER BY day)
FROM daily_sales;

EXPLAIN (COSTS OFF)
SELECT first_name, rank() OVER (PARTITION BY dept_id ORDER BY salary DESC)
FROM employees;
```

Output:

```
                       QUERY PLAN
---------------------------------------------------------
 WindowAgg
   ->  Index Scan using daily_sales_pkey on daily_sales
(2 rows)

               QUERY PLAN
----------------------------------------
 WindowAgg
   ->  Sort
         Sort Key: dept_id, salary DESC
         ->  Seq Scan on employees
(4 rows)
```

- In the first plan the primary key `(store, day)` (the columns that uniquely identify each row, which PostgreSQL backs with an index) already delivers rows in the needed order, so there's **no sort**. In the second, PostgreSQL must sort by `dept_id, salary DESC`.
- **Index tip:** an index on `(partition columns, order columns)` can remove the sort — e.g. `CREATE INDEX ON employees (dept_id, salary DESC)` (chapter 12).
- Windows with **different** `PARTITION BY`/`ORDER BY` need separate sorts; windows sharing the same definition are computed in one pass — another reason to reuse a named window.
- Sliding frames with `sum`/`count`/`avg` are computed incrementally (by adjusting the previous row's result rather than starting over), but `min`/`max` over a moving frame (and frames with `EXCLUDE`) may need re-scanning the frame, which gets expensive for large frames.
- A window over millions of rows sorts all of them; filter in `WHERE` first when the logic allows.

Chapter 8 TL;DR

- **Window function** = `f(...) OVER (PARTITION BY … ORDER BY … frame)`: computed per row over related rows; **row count never changes**.
- `PARTITION BY` = groups without collapsing → share of total, difference from group average.
- `ORDER BY` **in** `OVER` makes aggregates running; the default frame `RANGE UNBOUNDED PRECEDING … CURRENT ROW` includes ties — use a unique order or `ROWS`.

- **Frames**: `ROWS` (physical rows), `RANGE` (value distance, intervals, ties included), `GROUPS` (peer groups), `EXCLUDE` — PostgreSQL 11+ for offsets/`GROUPS`/`EXCLUDE`.
- **Ranking**: `row_number` (unique), `rank` (gaps), `dense_rank` (no gaps), `ntile`, `percent_rank`, `cume_dist`. Always decide how ties behave.
- **Value functions**: `lag`/`lead` (ignore frame), `first_value`/`last_value`/`nth_value` (use frame — give `last_value` a full frame). `IGNORE NULLS` only from PostgreSQL 19.
- **Patterns**: running total, moving average, growth vs `lag`, top-N per group (rank in CTE, filter outside), dedup (`row_number() = 1`), gaps & islands (`value − row_number()`, or flag + running sum).
- **Order of evaluation**: after `WHERE`/`GROUP BY`/`HAVING`, before `DISTINCT`/`ORDER BY`/`LIMIT`. No `QUALIFY` in PostgreSQL.
- **Performance**: one sort per distinct window; an index on (partition, order) columns can remove it.

Next: chapter 9 — DDL, DML and constraints: creating and changing tables, keys and constraints, and inserting, updating and deleting data safely.